SparkIIT AI Internship

Project 1 Report | NLP, Feature Extraction, Classification, Model Evaluation

Intelligent Document Classification System

Kaushikee Karmakar

Registered Email: kaushikeekarmakar@gmail.com

BCA, Asansol Engineering College

Program: SparkIIT AI Training and Internship (October 2026)

Deliverables: Python notebook (.ipynb) with outputs and this report

---

Contents

No.	Section
1	Abstract
2	Introduction
3	Objectives
4	Literature Review / Existing System
5	Methodology
6	Code
7	Output Snapshots
8	Results & Analysis
9	Conclusion

1. Abstract

Organizations receive large volumes of text, such as support tickets, news items and emails, that must be sorted by topic before anyone can act on them. This project builds and evaluates an AI-based document classification system using natural language processing and machine learning. Using 5,668 posts from six categories of the 20 Newsgroups corpus, the text was cleaned and tokenized, converted into TF-IDF features (unigrams and bigrams), and used to train three classifiers: Multinomial Naive Bayes, Logistic Regression and a Linear Support Vector Machine. The models were compared with Accuracy, Precision, Recall and F1-Score on a held-out test set of 1,131 documents. The Linear SVM performed best, and tuning its regularization strength with GridSearchCV gave a final accuracy of 91.07% and a weighted F1-score of 0.9105. A modern sentence-embedding baseline (all-MiniLM-L6-v2 with Logistic Regression) reached 0.8930 accuracy, below the TF-IDF model. Error analysis showed that most mistakes occur between topics with overlapping vocabulary, and that the model partly learns quirks of the 1990s data. Because the three TF-IDF models differ by about one percentage point, which is comparable to the sampling error of the test set, their ranking is suggestive rather than conclusive.

2. Introduction

Document classification is the task of automatically assigning a text document to one or more predefined categories. It is a core capability behind routing customer queries to the right team, organizing news feeds, filtering email and triaging large archives of documents. Doing this by hand does not scale, and simple keyword rules break down as soon as people phrase things differently.

Text is difficult for machine learning because it is high-dimensional (every distinct word is a potential feature), sparse, and ambiguous: the same word can signal different topics, and different topics can share vocabulary. A useful system therefore needs good text preprocessing, an informative numeric representation of text, a model suited to sparse data, and an honest evaluation.

This project covers that full workflow on a six-topic benchmark: collecting and preprocessing a text dataset, extracting features, training and comparing several classifiers, optimizing the best one, evaluating it with standard metrics, and analyzing where and why it fails. A modern embedding-based approach is included as a baseline so that the classical pipeline is judged against current practice.

3. Objectives

- Collect and preprocess a document dataset suitable for multi-class text classification.
- Perform text cleaning, tokenization and feature extraction.

- Develop AI-based classification models and train and test them with suitable algorithms.
- Evaluate performance using Accuracy, Precision, Recall and F1-Score.
- Analyze the results, identify the main sources of error, and optimize model performance.
- Compare the classical TF-IDF approach with a modern sentence-embedding baseline.
- Document the work as a reproducible, fully executed notebook with visible outputs.

4. Literature Review / Existing System

Manual and rule-based systems. Many organizations still sort documents with manual review or hand-written keyword rules. These are transparent but labor-intensive, hard to maintain, and brittle when wording changes.

Classical statistical classifiers. Representing text as weighted word counts (bag-of-words with TF-IDF weighting) and training Naive Bayes, Logistic Regression or Support Vector Machine classifiers is a long-standing and strong approach for topic classification, particularly when labelled data is limited. These models train quickly, run cheaply and are relatively easy to inspect, because the learned weights show which words drive each decision. Their weakness is that word order and meaning are ignored.

Dense embeddings. Word and sentence embeddings map text into dense vectors in which similar meanings lie close together, so a classifier can generalize across different wordings. Pre-trained sentence-encoder models (for example the sentence-transformers family) make this easy to apply, at the cost of a larger model and more computation.

Fine-tuned transformer models. Large pre-trained language models fine-tuned for classification are the most powerful option on many tasks, but need substantial compute, more engineering effort and heavier deployment.

Benchmark data. The 20 Newsgroups collection of Usenet posts is a widely used benchmark for text classification experiments and is distributed with scikit-learn.

Position of this project. This work builds a lightweight, transparent classical pipeline, evaluates it carefully (including its uncertainty), and compares it with an embedding baseline, so that the choice of approach is supported by evidence on the same data.

References. scikit-learn documentation, https://scikit-learn.org (text feature extraction, classifiers and the 20 Newsgroups dataset loader). Sentence-Transformers documentation, https://www.sbert.net (model all-MiniLM-L6-v2).

5. Methodology

5.1 Dataset

The 20 Newsgroups corpus was loaded through scikit-learn with six categories: comp.graphics, rec.autos, rec.sport.hockey, sci.med, sci.space and talk.politics.guns. Headers, footers and quoted replies were removed so that the model learns from the message content and not from metadata. The result is 5,668 documents, roughly balanced across the six classes (about 880 to 970 each). Posts are short: the median length is 85 words and the mean is 185 words.

5.2 Text preprocessing

Each document was lowercased, tokenized into alphabetic words of at least two letters, and stripped of English stop words. Documents that became empty after cleaning were removed. The data was split 80/20 with stratification, giving 1,131 test documents.

5.3 Feature extraction

TF-IDF features were computed for unigrams and bigrams (up to 20,000 features, minimum document frequency 2). The vectorizer was fitted on the training set only and then applied to the test set, which prevents information from the test data leaking into training.

5.4 Models

Three classifiers were trained on the same features: Multinomial Naive Bayes, Logistic Regression and a Linear Support Vector Machine (LinearSVC).

5.5 Evaluation

Models were evaluated on the held-out test set with Accuracy and support-weighted Precision, Recall and F1-Score, plus per-category metrics and a confusion matrix.

5.6 Optimization

The best model (Linear SVM) was tuned with GridSearchCV, using 3-fold cross-validation over the regularization strength C = 0.05, 0.1, 0.5, 1, 2 and 5, scored by weighted F1.

5.7 Embedding baseline

For comparison, every document was encoded with the pre-trained sentence-transformer all-MiniLM-L6-v2, and a Logistic Regression was trained on the resulting dense vectors using the same split.

5.8 Error analysis and testing on unseen text

Confused category pairs were extracted from the confusion matrix, the most influential words per category were read from the model weights, and six hand-written sentences outside the dataset were classified as a sanity check.

5.9 Tools

Python 3, scikit-learn, pandas, NumPy, Matplotlib, Seaborn, sentence-transformers, joblib, and Google Colab.

6. Code

The complete source code of the notebook is listed below, cell by cell, exactly as executed (very long lines are wrapped in this document). The same code, with all outputs visible, is in the submitted notebook *Project1_Document_Classification.ipynb*.

Cell 1

```
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix,
    classification_report

sns.set_theme(style="whitegrid")
print("Libraries loaded")
```

Cell 2

```
cats = ['sci.space', 'rec.sport.hockey', 'comp.graphics', 'talk.politics.guns', 'rec.autos', 'sci.med']
data = fetch_20newsgroups(subset='all', categories=cats, remove=('headers', 'footers', 'quotes'))
df = pd.DataFrame({'text': data.data, 'label': [data.target_names[i] for i in data.target]})
df = df[df['text'].str.strip().str.len() > 0].reset_index(drop=True)
print("Total documents:", len(df))
df.head()
```

Cell 3

```
plt.figure(figsize=(8, 4))
sns.countplot(y='label', data=df, order=df['label'].value_counts().index)
plt.title('Documents per category')
plt.tight_layout()
plt.savefig('fig_class_distribution.png', dpi=150)
plt.show()
print(df['text'].str.split().str.len().describe())
```

Cell 4

```
def clean_text(text):
    tokens = re.findall(r'[a-z]{2,}', text.lower())
    tokens = [t for t in tokens if t not in ENGLISH_STOP_WORDS]
    return ' '.join(tokens)

df['clean'] = df['text'].apply(clean_text)
df = df[df['clean'].str.len() > 0].reset_index(drop=True)
print("Before:", df['text'].iloc[0][:200])
print("After :", df['clean'].iloc[0][:200])
```

Cell 5

```
X_train, X_test, y_train, y_test = train_test_split(
    df['clean'], df['label'], test_size=0.2, random_state=42, stratify=df['label'])

tfidf = TfidfVectorizer(max_features=20000, ngram_range=(1, 2), min_df=2)
Xtr = tfidf.fit_transform(X_train)
Xte = tfidf.transform(X_test)
print("Train matrix:", Xtr.shape, "| Test matrix:", Xte.shape)
```

Cell 6

```
models = {
    'Naive Bayes': MultinomialNB(),
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'Linear SVM': LinearSVC(),
}
rows = []
for name, model in models.items():
    model.fit(Xtr, y_train)
    pred = model.predict(Xte)
    rows.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, pred),
        'Precision': precision_score(y_test, pred, average='weighted'),
        'Recall': recall_score(y_test, pred, average='weighted'),
        'F1-Score': f1_score(y_test, pred, average='weighted'),
    })
results = pd.DataFrame(rows).set_index('Model').round(4)
results
```

Cell 7

```
results.plot(kind='bar', figsize=(9, 5), ylim=(0.5, 1.0))
plt.title('Model comparison')
plt.xticks(rotation=0)
plt.legend(loc='lower right')
plt.tight_layout()
plt.savefig('fig_model_comparison.png', dpi=150)
plt.show()
```

Cell 8

```
base_f1 = results.loc['Linear SVM', 'F1-Score']
grid = GridSearchCV(LinearSVC(), {'C': [0.05, 0.1, 0.5, 1, 2, 5]}, cv=3, scoring='f1_weighted')
grid.fit(Xtr, y_train)
best = grid.best_estimator_
pred_best = best.predict(Xte)
tuned_f1 = f1_score(y_test, pred_best, average='weighted')
print("Best params:", grid.best_params_)
print(f"F1 before tuning: {base_f1:.4f} | after tuning: {tuned_f1:.4f}")
print(f"Tuned accuracy: {accuracy_score(y_test, pred_best):.4f}")
```

Cell 9

```
print(classification_report(y_test, pred_best))
classes = best.classes_
cm = confusion_matrix(y_test, pred_best, labels=classes)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=classes, yticklabels=classes)
plt.xlabel('Predicted'); plt.ylabel('Actual'); plt.title('Confusion matrix (tuned Linear SVM)')
plt.tight_layout()
plt.savefig('fig_confusion_matrix.png', dpi=150)
plt.show()
```

Cell 10

```
pairs = [(classes[i], classes[j], cm[i, j]) for i in range(len(classes)) for j in range(len(classes)) if i != j]
pairs = sorted(pairs, key=lambda x: -x[2])[:5]
print("Most confused pairs (actual -> predicted, count):")
for a, p, c in pairs:
    print(f"  {a} -> {p}: {c}")

feat = np.array(tfidf.get_feature_names_out())
print("\nTop words per category:")
for i, c in enumerate(classes):
    top = feat[np.argsort(best.coef_[i])[-8:][::-1]]
    print(f"  {c}: {', '.join(top)}")
```

Cell 11

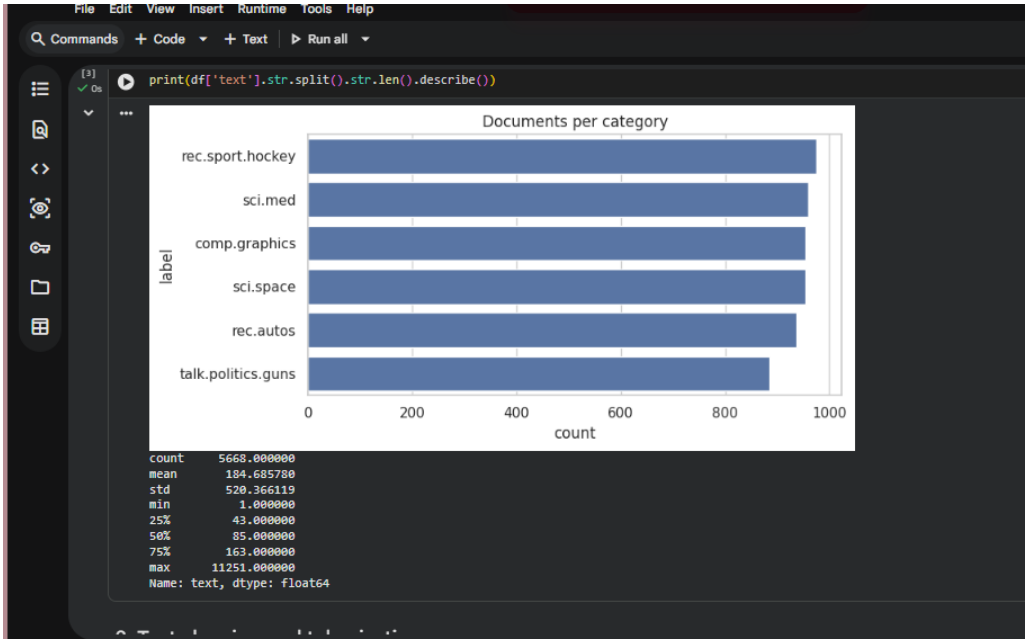
```
samples = [
    "NASA launched a new satellite to study the moon and Mars",
    "The goalie made an amazing save in overtime of the playoff game",
    "My new graphics card renders 3D scenes and images much faster",
    "The doctor prescribed antibiotics for the infection and fever",
    "This car has a powerful engine and great fuel mileage",
    "The debate over gun control laws continues in congress",
]
preds = best.predict(tfidf.transform([clean_text(s) for s in samples]))
for s, p in zip(samples, preds):
    print(f"[{p}]  {s}")
```

Cell 12

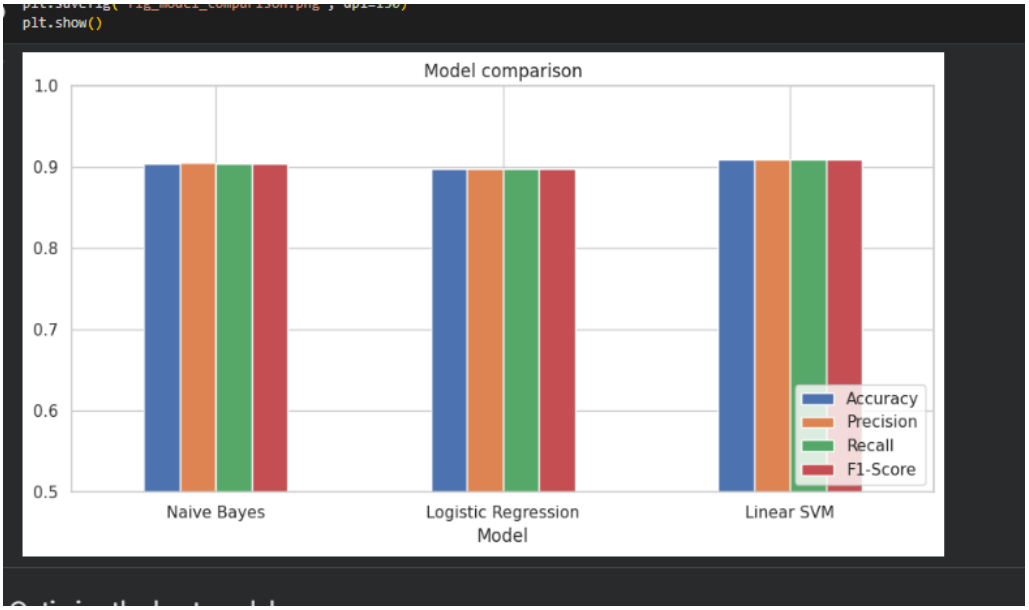
```
!pip -q install sentence-transformers
from sentence_transformers import SentenceTransformer
enc = SentenceTransformer('all-MiniLM-L6-v2')
Etr = enc.encode(df.loc[X_train.index, 'text'].tolist(), batch_size=64, show_progress_bar=True)
Ete = enc.encode(df.loc[X_test.index, 'text'].tolist(), batch_size=64)
emb = LogisticRegression(max_iter=2000).fit(Etr, y_train)
p = emb.predict(Ete)
print(f"Embeddings + LR | Accuracy: {accuracy_score(y_test, p):.4f} | F1: {f1_score(y_test, p,
    average='weighted'):.4f}")
```

7. Output Snapshots

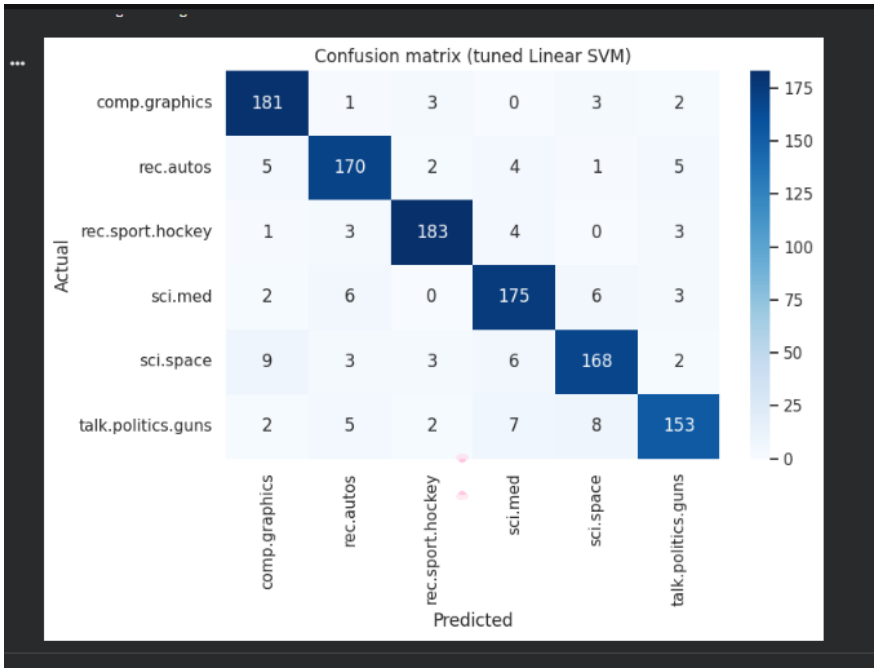
The snapshots below are taken from the executed Colab notebook: screenshots of charts and tables, followed by the printed output of the remaining cells.



Snapshot 1. Class distribution chart and document-length statistics (Colab output).



Snapshot 2. Model comparison chart (Colab output).



Snapshot 3. Confusion matrix of the tuned Linear SVM (Colab output).

Snapshot 4. Dataset size

```
Total documents: 5668
```

Snapshot 5. Model comparison table

```
                    Accuracy  Precision  Recall  F1-Score
Model
Naive Bayes           0.9045     0.9058  0.9045    0.9044
Logistic Regression   0.8974     0.8977  0.8974    0.8974
Linear SVM            0.9089     0.9090  0.9089    0.9088
```

Snapshot 6. Hyperparameter tuning result

```
Best params: {'C': 0.5}
F1 before tuning: 0.9088 | after tuning: 0.9105
Tuned accuracy: 0.9107
```

Snapshot 7. Error analysis

```
Most confused pairs (actual -> predicted, count):
  sci.space -> comp.graphics: 9
  talk.politics.guns -> sci.space: 8
  talk.politics.guns -> sci.med: 7
  sci.med -> rec.autos: 6
  sci.med -> sci.space: 6

Top words per category:
  comp.graphics: graphics, image, file, files, computer, version, pov, program
  rec.autos: car, cars, engine, dealer, ford, oil, radar, bmw
  rec.sport.hockey: game, hockey, team, games, nhl, season, puck, roger
  sci.med: doctor, medical, health, msg, disease, treatment, photography, blood
  sci.space: space, orbit, shuttle, launch, moon, earth, sky, nasa
  talk.politics.guns: gun, guns, weapons, fbi, firearms, government, koresh, house
```

Snapshot 8. Predictions on unseen sentences

```
[sci.space]  NASA launched a new satellite to study the moon and Mars
[rec.sport.hockey]  The goalie made an amazing save in overtime of the playoff game
[comp.graphics]  My new graphics card renders 3D scenes and images much faster
[sci.med]  The doctor prescribed antibiotics for the infection and fever
[rec.autos]  This car has a powerful engine and great fuel mileage
[talk.politics.guns]  The debate over gun control laws continues in congress
```

Snapshot 9. Sentence-embedding baseline

```
Embeddings + LR | Accuracy: 0.8930 | F1: 0.8930
```

8. Results & Analysis

Model	Accuracy	Precision	Recall	F1-Score
Naive Bayes	0.9045	0.9058	0.9045	0.9044
Logistic Regression	0.8974	0.8977	0.8974	0.8974
Linear SVM	0.9089	0.9090	0.9089	0.9088
Linear SVM (tuned, C = 0.5)	0.9107	0.9108	0.9107	0.9105
MiniLM embeddings + Logistic Regression	0.8930	n/a	0.8930	0.8930

Table 1. Test-set results (precision, recall and F1 are support-weighted). Tuned-model precision and recall are computed from its confusion matrix; precision for the embedding baseline was not recorded.

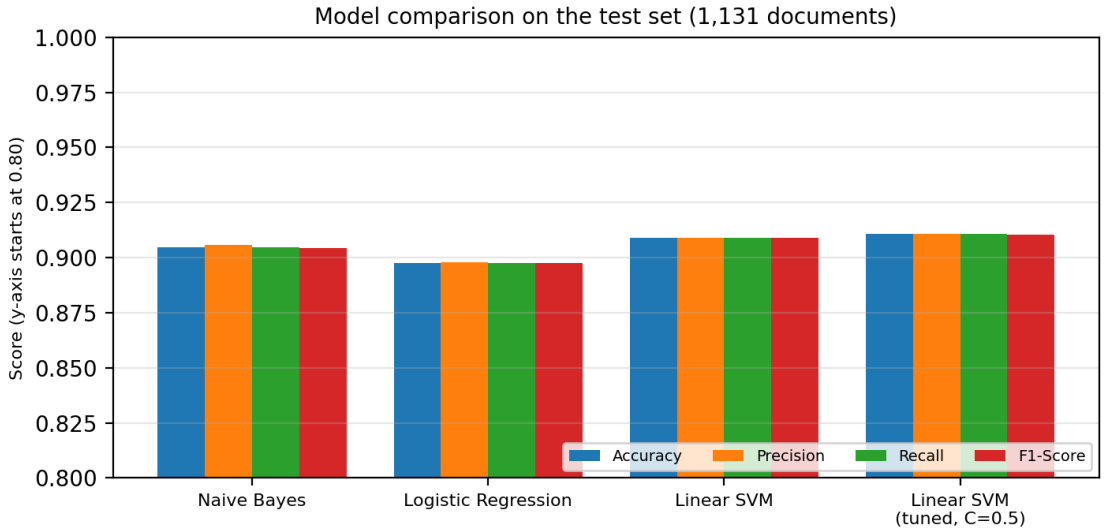


Figure 1. Model comparison on the test set (y-axis starts at 0.80).

Category	Precision	Recall	F1-Score	Support
comp.graphics	0.905	0.953	0.928	190
rec.autos	0.904	0.909	0.907	187
rec.sport.hockey	0.948	0.943	0.946	194
sci.med	0.893	0.911	0.902	192
sci.space	0.903	0.880	0.891	191
talk.politics.guns	0.911	0.864	0.887	177

Table 2. Per-category performance of the tuned Linear SVM (computed from the confusion matrix).

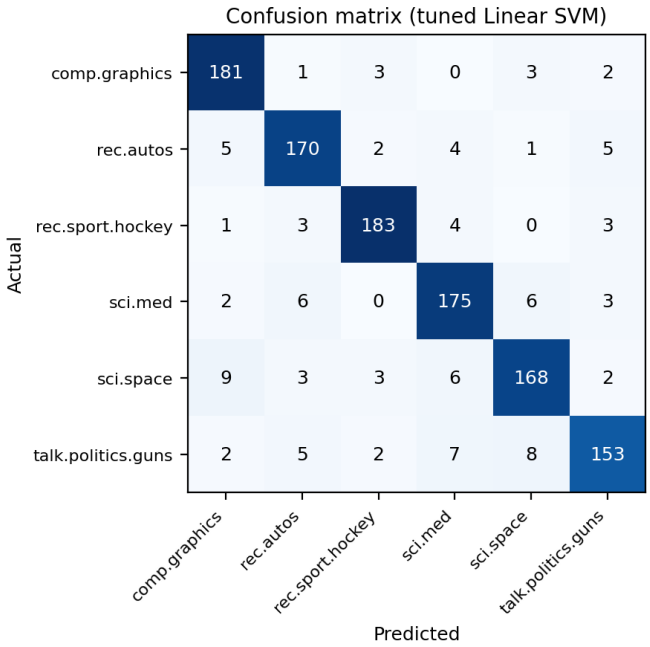


Figure 2. Confusion matrix of the tuned Linear SVM.

Analysis

- **Model comparison.** All three TF-IDF models finish within about 1.2 points of each other. With 1,131 test documents the standard error of the accuracy is roughly 0.8 points, so the ranking (Linear SVM, then Naive Bayes, then Logistic Regression) is suggestive rather than statistically proven. Tuning raised F1 from 0.9088 to 0.9105, a gain too small to call meaningful. The honest conclusion is that TF-IDF with a linear model works well on this task and the exact choice of linear model matters little.
- **Embedding baseline.** Sentence embeddings with an untuned Logistic Regression reached 0.8930 accuracy, about 1.8 points below the tuned TF-IDF model. Plausible reasons are that these posts are short and driven by topic keywords, which TF-IDF captures directly, and that general-purpose embeddings are built for sentence similarity rather than these specific topics. These explanations were not tested, and the result comes from a single split. Embeddings were also more expensive: encoding the 4,500 training documents took about 6 minutes 44 seconds on the Colab runtime used, versus seconds for TF-IDF.
- **Strongest and weakest categories.** rec.sport.hockey (F1 0.946) and comp.graphics (F1 0.928) are the easiest because they have very distinctive vocabulary. talk.politics.guns has the lowest recall (0.864) and sci.space the lowest F1 (0.891).
- **Confusions.** The most frequent errors were sci.space predicted as comp.graphics (9), talk.politics.guns predicted as sci.space (8) and as sci.med (7). A likely explanation is vocabulary overlap, for example space posts discussing images and files resembling graphics posts. This was not verified on the individual documents.
- **What the model learned.** The top words are sensible (gun, firearms, weapons; orbit, shuttle, nasa; puck, nhl, season; car, engine, dealer). Some are corpus-specific quirks: 'koresh' (the 1993 Waco standoff), 'roger' (probably a frequently mentioned name) and 'msg' or 'photography' under sci.med. The model therefore partly learns the habits of this 1990s Usenet data and may not transfer directly to modern text.
- **Unseen text.** All six hand-written test sentences were classified correctly. With only six examples this is a sanity check, not a performance estimate.
- **Limitations.** A single train/test split was used for the final numbers, only six topics and old text were covered, and TF-IDF ignores word order and meaning.

9. Conclusion

The project delivered a complete, leakage-free NLP classification pipeline that reaches about 91% accuracy and 0.91 weighted F1 across six topic categories. It compared three classical models and a sentence-embedding baseline, tuned the best model, evaluated it with the required metrics, and analyzed its errors honestly, including its dependence on this particular dataset. The classical TF-IDF and Linear SVM combination was both the most accurate and the cheapest to run here, which makes it the sensible choice for this task.

Future work. Use k-fold cross-validation for the final estimates, test on modern and noisier documents with more categories, try fine-tuning a transformer model, and wrap the classifier in a small web application so users can paste a document and see the predicted category with a confidence score.